



الجامعة الإسلامية - غزة
كلية تكنولوجيا المعلومات
ويب 2 عملي
SDEV 2106, WDMM 2010,
MOBC 2102
الفصل الثاني: 2025/2026
المهندس: محمد زقلام



ClinicDesk

Clinic Management Dashboard

PHP Final Project

SDEV 2106 / WDMM 2010 / MOBC 2102 · Semester 2, 2025–2026

Eng: Mohammed Zuqlam

1. Project Overview & Context

ClinicDesk is a private, login-protected clinic management dashboard. There are no public pages — every route requires an authenticated session. The application serves three types of users: administrators who manage the system, doctors who manage their appointments and prescriptions, and patients who book appointments and track their medical history.

Role	Can Do	Restricted From
Admin	Manage all users, doctors & specializations. View all appointments & prescriptions. Generate all reports. Activate/deactivate accounts.	Cannot impersonate other users or access prescription file contents directly.
Doctor	View own appointment schedule. Update appointment status (confirm/complete/cancel). Add prescriptions with file upload. Edit own profile.	Cannot view other doctors' appointments. Cannot manage users.
Patient	Book appointments (with conflict check). View own appointment history and status. Download own prescriptions. Edit own profile.	Cannot see other patients' data. Cannot change appointment status.

All three roles log in through the same login page. After authentication the application reads the role from the session and redirects to the appropriate dashboard. There is no separate URL per role.

2. Learning Objectives

Every item below maps to a specific part of the project. By submission you will have applied each concept in working, reviewed code.

- **Singleton Pattern** A single Database class instance is created once and reused across all models — never instantiated twice.
- **OOP Models** Each database table has a dedicated Model class extending a shared BaseModel. No raw SQL lives in controllers or views.
- **Prepared Statements** Every query — read or write — uses mysqli prepared statements. Zero string interpolation into SQL.
- **Session-Based Auth** Login creates a secure session. Every page checks authentication before rendering. `session_regenerate_id()` is called on login.
- **Role-Based Access** `Auth::requireRole()` is called at the top of every controller action. Resource ownership is verified separately.
- **CSRF Protection** Every POST form carries a hidden token. Every POST handler validates it with `hash_equals()` before processing.
- **Image & File Upload** Avatars and doctor photos use `getimagesize()` validation. Prescription PDFs use `finfo_file()`. Files are served through PHP, never directly.
- **Pagination** All listing pages (appointments, users) are paginated using LIMIT/OFFSET in prepared statements via a reusable Paginator class.
- **Search & Filter** Appointments can be filtered by doctor, date range, and status. The WHERE clause is built dynamically in PHP.
- **Dashboard Statistics** Each dashboard shows role-relevant aggregate data queried with COUNT(), GROUP BY, and JOIN in a single efficient query.
- **Reports** The admin can generate appointment reports filtered by date range and doctor, and export them as a CSV file using `fputcsv()`.
- **Reusable Partial**s Header, sidebar, footer, and flash alerts are extracted into include files used by every view.

3. Template: AdminLTE 3 Integration Guide

Because ClinicDesk is a pure dashboard with no public pages, you will use a single template throughout: AdminLTE 3. This is the most widely used open-source PHP admin template in the industry and integrating it is a directly transferable skill.

Download & Setup

URL: <https://adminlte.io> — download the free v3.x release (MIT licence)

Extract to: `public/assets/adminlte/` — keep the internal folder structure intact

How to split AdminLTE into PHP partials

Open the AdminLTE starter.html file. You will identify four zones and map each to a partial:

AdminLTE Zone	Your PHP Partial File
<code><head></code> + CSS links	<code>views/partials/header.php</code> — add <code><?php echo \$pageTitle ?></code> in <code><title></code>
<code><nav></code> top bar	<code>views/partials/navbar.php</code> — PHP conditions show role-specific links
<code><aside></code> sidebar	<code>views/partials/sidebar.php</code> — highlight active link based on current page
Closing JS scripts	<code>views/partials/footer.php</code> — all <code><script></code> tags go here, never mid-page

Every view file follows this structure:

```
<?php
require_once __DIR__ . '/../core/Auth.php';
Auth::requireRole("doctor"); // role check FIRST

$pageTitle = "My Schedule";
require_once __DIR__ . '/../partials/header.php';
require_once __DIR__ . '/../partials/navbar.php';
require_once __DIR__ . '/../partials/sidebar.php';
?>

<!-- AdminLTE .content-wrapper -->
<div class="content-wrapper">
    <section class="content">
        <!-- YOUR DYNAMIC PHP CONTENT HERE -->
    </section>
</div>

<?php require_once __DIR__ . '/../partials/footer.php'; ?>
```

Key AdminLTE components to use in this project:

- .info-box or .small-box — dashboard statistic cards (appointments today, total patients, etc.)
- .card — wraps every form (book appointment, add prescription, edit profile)
- DataTables plugin (included in AdminLTE) — for all listing tables with built-in client-side search
- .breadcrumb in .content-header — shows the user where they are (Dashboard > Appointments > New)
- Bootstrap badges — colour-coded appointment status (pending=warning, confirmed=info, completed=success, cancelled=danger)
- views/partials/alerts.php — reads \$_SESSION["flash"] and renders as AdminLTE alert-* divs, then unsets the key

Do not link any external CDN in your views. All CSS and JS must load from your local public/assets/adminlte/ folder. This ensures the project works offline during the demonstration.

4. Database Schema

Create a MySQL database named clinicdesk_db. Execute the five CREATE TABLE statements below in the exact order shown — foreign key constraints require parent tables to exist first.

Table 1 — users

```
CREATE TABLE users (  
  id      INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  name    VARCHAR(120) NOT NULL,  
  email   VARCHAR(180) NOT NULL UNIQUE,  
  password VARCHAR(255) NOT NULL,  
  role    ENUM("admin","doctor","patient") NOT NULL DEFAULT "patient",  
  phone   VARCHAR(20)  DEFAULT NULL,  
  avatar  VARCHAR(255)  DEFAULT NULL,  
  is_active TINYINT(1) NOT NULL DEFAULT 1,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;  
  
-- Seed: one admin account  
INSERT INTO users (name, email, password, role)  
VALUES ("Admin", "admin@clinic.local",  
       "$2y$12$...", "admin"); -- replace hash with password_hash("Admin@1234")
```

Table 2 — specializations

```
CREATE TABLE specializations (  
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL UNIQUE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
INSERT INTO specializations (name) VALUES  
  ("General Practice"), ("Cardiology"), ("Dermatology"),  
  ("Pediatrics"), ("Orthopedics"), ("Neurology"),  
  ("Ophthalmology"), ("ENT"), ("Psychiatry");
```

Table 3 — doctors

```
CREATE TABLE doctors (  
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  user_id INT UNSIGNED NOT NULL UNIQUE,  
  specialization_id INT UNSIGNED NOT NULL,  
  bio TEXT DEFAULT NULL,  
  consultation_fee DECIMAL(8,2) NOT NULL DEFAULT 0.00,  
  available_days VARCHAR(50) NOT NULL DEFAULT "Sun,Mon,Tue,Wed,Thu",  
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
  FOREIGN KEY (specialization_id) REFERENCES specializations(id) ON DELETE RESTRICT  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```
-- Note: available_days stores a comma-separated string e.g. "Sun,Mon,Wed"
```

```
-- explode(",", $doctor["available_days"]) in PHP to get an array for checking.
```

Table 4 — appointments

```
CREATE TABLE appointments (  
  id      INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  patient_id INT UNSIGNED NOT NULL,  
  doctor_id INT UNSIGNED NOT NULL,  
  appt_date DATE      NOT NULL,  
  appt_time TIME      NOT NULL,  
  status   ENUM("pending","confirmed","completed","cancelled")  
           NOT NULL DEFAULT "pending",  
  reason   VARCHAR(255) DEFAULT NULL,  
  doctor_notes TEXT     DEFAULT NULL,  
  created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  UNIQUE KEY no_double_booking (doctor_id, appt_date, appt_time),  
  FOREIGN KEY (patient_id) REFERENCES users(id) ON DELETE CASCADE,  
  FOREIGN KEY (doctor_id) REFERENCES doctors(id) ON DELETE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Table 5 — prescriptions

```
CREATE TABLE prescriptions (  
  id          INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,  
  appointment_id INT UNSIGNED NOT NULL UNIQUE, -- one prescription per appointment  
  diagnosis   TEXT      NOT NULL,  
  medications TEXT      NOT NULL,  
  notes       TEXT      DEFAULT NULL,  
  file_path   VARCHAR(255) DEFAULT NULL, -- optional scanned PDF upload  
  created_at  TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (appointment_id) REFERENCES appointments(id) ON DELETE CASCADE  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Entity relationships at a glance:

- users → doctors: one-to-one (a doctor user has exactly one doctors record)
- specializations → doctors: one-to-many (a specialization has many doctors)
- users (patient) → appointments: one-to-many (a patient has many appointments)
- doctors → appointments: one-to-many (a doctor has many appointments)
- appointments → prescriptions: one-to-one (one appointment, at most one prescription)
- UNIQUE KEY on (doctor_id, appt_date, appt_time) enforces no double-booking at DB level

5. Project Structure

All files must follow the hierarchy below exactly. This structure enforces the separation of config, core logic, models, controllers, and views.

```
clinicdesk/
├── index.php          ← Front controller: routes via $_GET["page"] & ["action"]
├── .htaccess          ← Forces all requests through index.php
├── README.md
├──
├── config/
│   ├── config.php    ← APP_NAME, BASE_URL, ITEMS_PER_PAGE, upload limits
│   └── database.php  ← DB host, name, user, pass — excluded from git
├──
├── core/
│   ├── Database.php  ← Singleton mysqli wrapper with query() helper
│   ├── Auth.php      ← login(), logout(), check(), currentUser(), requireRole()
│   ├── CSRF.php      ← generateToken(), validateToken()
│   ├── Paginator.php ← totalPages(), hasPrev(), hasNext(), offset()
│   └── helpers.php    ← redirect(), sanitize(), formatDate(), formatTime()
├──
├── models/
│   ├── BaseModel.php ← Abstract: holds $db, shared execute() method
│   └── UserModel.php
```

```
| └─ DoctorModel.php
| └─ AppointmentModel.php
| └─ PrescriptionModel.php
| └─ SpecializationModel.php
|
└─ controllers/
| └─ AuthController.php    ← login/logout POST handling
| └─ DashboardController.php ← stats queries, routes to role dashboard view
| └─ UserController.php    ← Admin: CRUD users
| └─ DoctorController.php  ← Admin+Doctor: manage doctor records
| └─ AppointmentController.php ← All roles: book/view/update appointments
| └─ PrescriptionController.php ← Doctor: add prescription; Patient: download
| └─ ReportController.php   ← Admin: generate + export CSV reports
|
└─ views/
| └─ partials/
| | └─ header.php
| | └─ navbar.php
| | └─ sidebar.php        ← Role-aware navigation links
| | └─ footer.php
| | └─ alerts.php         ← Flash message renderer
| └─ auth/
| | └─ login.php
| └─ dashboard/
| | └─ admin.php
| | └─ doctor.php
| | └─ patient.php
| └─ users/                ← Admin: list, create, edit user views
| └─ doctors/              ← Admin+Doctor: list, create, edit doctor views
| └─ appointments/        ← All: book, list, detail, edit views
| └─ prescriptions/       ← Doctor: add; Patient: view
| └─ reports/
| | └─ index.php
| └─ errors/
| └─ 403.php
```

```
|   └─ 404.php
|
└─ public/
    └─ assets/adminlte/    ← AdminLTE CSS/JS/fonts (from download)
        └─ uploads/
            └─ avatars/    ← User avatars (JPEG/PNG, max 1MB)
            └─ doctor_photos/ ← Doctor profile photos
            └─ prescriptions/ ← Prescription PDFs — NOT directly accessible
```

```
# public/uploads/prescriptions/.htaccess
Order Deny,Allow
Deny from all
```

The front controller pattern: every request goes to index.php. It reads `$_GET["page"]` (e.g. "appointments") and `$_GET["action"]` (e.g. "book") and requires the matching controller. This keeps URLs clean and centralises the routing logic.

6. Part 1 — Core Classes

Build these four classes before writing a single model or controller. Everything else depends on them.

core/Database.php — Singleton

- Private static `$instance` property. Private constructor calls `new mysqli()` using constants from `config/database.php`.
- Static `getInstance()`: if `$instance` is null, create it. Return `$instance` always.
- Public `query(string $sql, string $types="", array $params=[])`: executes a prepared statement. Binds params with ... spread operator. Returns `get_result()` for SELECT, true/false for others.
- Public `lastInsertId()`: returns `$this->conn->insert_id`.
- Throw a `RuntimeException` with a safe message (not the real error) if the connection fails.

core/Auth.php

- `login(array $user)`: stores id, name, role in `$_SESSION["user"]`. Calls `session_regenerate_id(true)`.
- `logout()`: calls `session_unset()` and `session_destroy()`. Redirects to login page.
- `check()`: bool — returns true if `$_SESSION["user"]` is set.
- `currentUser()`: ?array — returns `$_SESSION["user"]` or null.
- `role()`: string — returns `$_SESSION["user"]["role"]` or empty string.
- `requireRole(string ...$roles)`: if `!check()`, redirect to login. If `role()` not in `$roles`, redirect to 403 page.

core/CSRF.php

- `generateToken()`: creates `bin2hex(random_bytes(32))`, stores in `$_SESSION["csrf_token"]`, returns it.
- `validateToken(string $token)`: bool — uses `hash_equals($_SESSION["csrf_token"], $token)`. Returns false if session key missing.
- Usage pattern: every POST form includes `<input type="hidden" name="csrf_token" value="<?= CSRF::generateToken() ?>">`.
- Every POST handler calls `CSRF::validateToken($_POST["csrf_token"] ?? "")` or redirect with error.

core/Paginator.php

- Constructor: `__construct(int $totalItems, int $perPage, int $currentPage)`.
- `offset()`: int — returns `($currentPage - 1) * $perPage`. Used directly in the `LIMIT ? OFFSET ?` query.

- `totalPages(): int` — returns `(int) ceil($totalItems / $perPage)`.
 - `hasPrev()` / `hasNext()`: bool comparisons on current page vs bounds.
 - A paginator partial view renders the Bootstrap pagination component using these methods.
-

7. Part 2 — Models & Manager Classes

One Model per table. Models talk to the database. Controllers talk to Models. Views never talk to the database.

models/BaseModel.php

- Abstract class. Constructor: `$this->db = Database::getInstance()`.
- Protected `execute(string $sql, string $types="", array $params=[])`: delegates to `$this->db->query()`. Centralises error handling for all child models.
- Every child model calls `$this->execute()` — never `$this->db->query()` directly. This allows you to swap error handling in one place.

models/UserModel.php

- `findById(int $id): ?array`
- `findByEmail(string $email): ?array` — used during login.
- `create(array $data): int` — inserts and returns new ID. Password must already be hashed before calling.
- `update(int $id, array $data): bool` — allow updating name, phone, avatar.
- `updatePassword(int $id, string $newHash): bool`
- `getAllPaginated(int $page, string $role="")`: array — filter by role if provided.

- `countAll(string $role="")`: int
- `toggleActive(int $id)`: bool — flips `is_active` between 0 and 1.

models/DoctorModel.php

- `findByUserId(int $userId)`: ?array — JOIN with users and specializations.
- `getAll()`: array — for dropdown lists (book appointment form).
- `getAllPaginated(int $page)`: array — JOIN with users + specializations for admin list.
- `create(array $data)`: int
- `update(int $doctorId, array $data)`: bool
- `getAvailableDays(int $doctorId)`: array — returns exploded `available_days` array.

models/AppointmentModel.php

- `book(array $data)`: bool — INSERT. Returns false if UNIQUE constraint fails (double-booking attempt).
- `hasConflict(int $doctorId, string $date, string $time)`: bool — SELECT before booking to show friendly error.
- `getByPatient(int $patientId, int $page, array $filters)`: array
- `getByDoctor(int $doctorId, int $page, array $filters)`: array
- `getAll(int $page, array $filters)`: array — admin only, JOIN patients + doctors.
- `countFiltered(string $scope, int $scopeId, array $filters)`: int — used by Paginator.
- `updateStatus(int $id, string $status, string $notes="")`: bool — doctor/admin action.
- `findById(int $id)`: ?array — include patient and doctor info via JOIN.

models/PrescriptionModel.php

- findByAppointmentId(int \$apptId): ?array
- create(array \$data): int — includes optional file_path.
- update(int \$id, array \$data): bool
- getByPatient(int \$patientId): array — JOIN appointments to verify ownership.

8. Part 3 — Authentication (The Login Wall)

There is no registration page. The Admin creates all accounts. This is realistic for internal clinic software and simplifies your auth work significantly.

Login page (GET)

- If Auth::check() is already true, redirect to dashboard immediately.
- Render views/auth/login.php — a clean centred AdminLTE login card.
- The form includes a hidden CSRF token. Email and password fields only.
- No "Register" link anywhere in the UI.

Login handler (POST)

- Validate CSRF token first. If invalid: flash error, redirect back.
- Sanitize email (filter_var FILTER_SANITIZE_EMAIL).
- UserModel::findByEmail(). If not found: flash generic error "Invalid credentials" (never say which field is wrong).
- Check is_active === 1. If not: flash "Account suspended. Contact admin."

- `password_verify($submitted, $user["password"])`. If fail: flash generic error.
- `Auth::login($user)`. Redirect to `index.php?page=dashboard`.

Logout handler (POST)

- Logout must be a POST form with CSRF, not a GET link. A GET link can be triggered by a `<img src>` tag in an email — a real attack.
- Validate CSRF, call `Auth::logout()`, redirect to login.

Admin: create accounts

- Admin can create patient and doctor accounts from the Users panel.
- A temporary password is set by admin. Patient/doctor should change it on first login (implement `first_login` flag as a challenge).
- When creating a doctor account, the admin is also prompted to fill in the doctors table fields (specialization, fee, available days) in the same form submission.

9. Part 4 — Admin Panel

The admin role has full system oversight. Every admin action requires both `Auth::requireRole("admin")` and a valid CSRF token.

User Management

- Paginated list of all users with role filter and search by name/email.
- Create user: name, email, temp password (hashed), role, phone.

- Edit user: name, phone, is_active toggle. Password is changed separately.
- Deactivate/Activate: toggleActive() flips the flag. Deactivated users are rejected at login.
- A user cannot deactivate their own account (check currentUser()["id"] !== \$targetId).

Doctor Management

- List all doctors with their user name, specialization, fee, and availability.
- Edit doctor record: specialization_id (dropdown from SpecializationModel::getAll()), bio, consultation_fee, available_days (checkboxes for Sun–Thu/Fri/Sat).
- On save, implode the checked days array back to a comma-separated string for storage.
- Upload doctor profile photo: JPEG/PNG only, max 1 MB, stored in public/uploads/doctor_photos/. Replace old file on update.

Specialization Management

- Simple CRUD: list all, add new, delete. No pagination needed (small dataset).
- Before deleting a specialization, check if any doctors use it (COUNT query). If yes, show error and block deletion.
- This is a good example of a manager-style method: SpecializationModel::isSafeToDelete(int \$id): bool.

10. Part 5 — Appointments

Appointments are the core entity. Three roles interact with them in different ways. All filtering uses dynamic WHERE clause construction — no if/else query switching.

Patient: Book Appointment

- Form fields: doctor (dropdown), preferred date (date input), time slot (select from fixed slots: 09:00, 09:30, 10:00 ... 16:00), reason (short text).
- On doctor select, show their available_days so the patient knows which days are valid. Hint: load this with a second query after doctor selection, or display it statically below the dropdown.
- Conflict check: AppointmentModel::hasConflict(\$doctorId, \$date, \$time). If true: flash "This slot is already booked, please choose another time."
- Also validate: \$date must not be in the past, and \$date day-of-week must be in the doctor's available_days.
- On success: insert appointment with status="pending", flash success, redirect to my appointments.

Patient: My Appointments

- Paginated table: Doctor, Specialization, Date, Time, Status (badge), Reason, Action.
- Filter bar: status dropdown (all/pending/confirmed/completed/cancelled), date range.
- Action column: "View Prescription" link (only if status=completed and prescription exists). "Cancel" button (only if status=pending — POST form with CSRF).

Doctor: My Schedule

- Today's appointments shown prominently at the top as a separate query.
- Full paginated list below with same filter bar.
- Per-appointment actions: Confirm (pending→confirmed), Complete (confirmed→completed), Cancel (pending/confirmed→cancelled).
- On Complete: a "Add Prescription" button appears — links to prescription creation form.

- Doctor can add notes to any of their appointments (doctor_notes field).

Admin: All Appointments

- Paginated table with filters: doctor (dropdown), patient name (text search), date range, status.
- Admin can update any appointment status and view all details.
- Dynamic WHERE clause: build \$conditions = [] and \$params = []. For each active filter, push a condition string and its value. Join with implode(" AND ", \$conditions). If empty, WHERE clause is omitted.

11. Part 6 — Prescriptions & File Upload

Prescriptions are added by doctors after completing an appointment. A scanned PDF may be attached. Access to prescription files is strictly controlled.

Doctor: Add Prescription

- Only accessible if: Auth role = doctor AND the appointment belongs to this doctor AND status = completed AND no prescription exists yet.
- Form fields: diagnosis (textarea), medications (textarea), notes (textarea, optional), prescription_file (file input, optional).
- File upload rules: PDF only. Validate with finfo_file() — check for application/pdf MIME. Max 3 MB. Filename: prescription_{appointmentId}_{timestamp}.pdf. Store in public/uploads/prescriptions/.
- On success: insert prescription record, flash success, redirect to appointment detail.

Secure file serving

- The prescriptions/ folder is blocked from direct URL access via .htaccess.
- A controller action download_prescription reads the appointment ID from \$_GET["id"].
- Verify: user is logged in. If patient: prescription's appointment patient_id must match session user id. If doctor: appointment doctor_id must match. Admin: always allowed.
- If authorised: read the file_path from DB, then use readfile() to stream it with headers: Content-Type: application/pdf and Content-Disposition: attachment; filename="prescription.pdf".
- If not authorised: 403 page. If file missing on disk: flash error and redirect.

Patient: View Prescriptions

- A "My Prescriptions" page lists all prescriptions for completed appointments.
- Columns: Doctor, Date, Diagnosis (truncated), Download PDF (only if file_path is not null).
- The Download link hits the secure controller action above — never a direct file URL.

12. Part 7 — Dashboard Statistics & Reports

Statistics are queried fresh on every dashboard load. Never hardcode numbers. Reports are admin-only and exportable.

Admin Dashboard Stats

- Total users by role: SELECT role, COUNT(*) as total FROM users GROUP BY role — one query, loop the result in PHP.

- Appointments today: `SELECT COUNT(*) FROM appointments WHERE appt_date = CURDATE();`
- Appointments this week by status: `SELECT status, COUNT(*) FROM appointments WHERE WEEK(appt_date) = WEEK(NOW()) GROUP BY status.`
- Display as AdminLTE .small-box components with colour per role/status.
- Recent 5 appointments table below stats: `JOIN patients + doctors, ORDER BY created_at DESC LIMIT 5.`

Doctor Dashboard Stats

- Today's appointments list (the most important thing a doctor wants to see).
- Counts: total appointments this month, pending, completed.
- Upcoming: next 5 appointments ordered by date/time.

Patient Dashboard Stats

- Active appointments (pending + confirmed): count and list.
- Total completed appointments. Total prescriptions available.
- Next upcoming appointment card displayed prominently at top.

Admin Reports

- Filter form: start_date, end_date (required), doctor_id (optional dropdown), status (optional).
- Result table: Patient Name, Doctor Name, Specialization, Date, Time, Status, Reason.
- Summary row at bottom: total shown, grouped status counts.
- Validate: start_date <= end_date. Both required.

- Export CSV: when `$_GET["export"] === "csv"`, set Content-Type: text/csv + Content-Disposition headers, write header row then data rows using `fputcsv()` to `php://output`. `exit()` after.

13. Security Checklist

Every item in this table is graded. Review it before submission.

Item	Requirement
Passwords	<code>password_hash(PASSWORD_BCRYPT)</code> only. <code>password_verify()</code> for login. Zero MD5/SHA1.
CSRF	Every POST form has a hidden token. Every handler calls <code>CSRF::validateToken()</code> with <code>hash_equals()</code> .
Prepared Statements	Zero string concatenation into SQL. Every query uses ? placeholders and bound params.
XSS	Every echo of user data: <code>htmlspecialchars(\$val, ENT_QUOTES, "UTF-8")</code> . No exceptions.
Session fixation	<code>session_regenerate_id(true)</code> immediately after <code>Auth::login()</code> .
File type validation	Images: <code>getimagesize()</code> must return a valid image. PDFs: <code>finfo_file()</code> must return <code>application/pdf</code> .
File access	Prescriptions folder blocked via <code>.htaccess</code> . Files served only through ownership-checked PHP action.
Directory listing	Each uploads subfolder contains an <code>index.php</code> that calls <code>exit()</code> or a <code>deny .htaccess</code> .
Error display	<code>display_errors</code> Off in <code>php.ini/config</code> . Errors logged, never shown to users.

DB credentials in git	.gitignore includes config/database.php. Verify with git status before every push.
Role enforcement	Auth::requireRole() is the FIRST line of every controller action. Never rely on hidden UI.

14. Advanced Challenge Features

These challenges are realistic features found in real clinic software. Implement them only after all core parts are fully working.

Doctor Availability Calendar

Replace the plain appointments list for doctors with a weekly calendar view. Build a 7-column HTML table where each column is a day of the week. Query the doctor's appointments for the current week and place them in the correct cell. Highlight today's column. Navigation buttons move to the previous/next week using a week offset in the URL (?week=-1, ?week=1).

First-Login Password Change Enforcement

Add a first_login TINYINT(1) column to users (default 1). After login, if first_login === 1, redirect to a mandatory change-password page before any other page. The change-password form requires: current password (verify with password_verify), new password (min 8 chars, mixed case + number), confirm new password. On success: update hash, set first_login = 0, redirect to dashboard.

Appointment Cancellation Reason & Audit Log

When an appointment is cancelled by any role, require a cancellation reason (textarea, min 10 chars). Add a cancelled_by and cancellation_reason column to appointments. Create a separate appointment_logs

table: id, appointment_id, changed_by_user_id, old_status, new_status, note, changed_at. Every status change writes a row to this log. Admin can view the full log for any appointment.

Patient Self-Registration with Admin Approval

Add a public login page that also has a "Request Account" link. The request form collects: name, email, phone, and a short reason. Submissions are stored in a pending_registrations table with status="pending". Admin sees a "Pending Requests" count badge in the sidebar. Admin can approve (which creates the actual user account and emails a temp password) or reject (with a reason that is shown if the patient tries to request again with the same email).

JSON API Endpoint for Appointments

Create a read-only JSON API at index.php?page=api&action=appointments. Authentication uses a simple API token stored in the users table (api_token column, generated with bin2hex(random_bytes(16))). The request must send the token in the Authorization: Bearer {token} HTTP header. Read it with `getallheaders()["Authorization"]`. Validate, look up user, check role. Return a JSON-encoded array of appointments. Support `?status=` and `?date=` query parameters. Set Content-Type: application/json.

Dashboard Chart with Chart.js

On the Admin dashboard, add a bar chart showing the number of appointments per day for the last 14 days. Query: `SELECT appt_date, COUNT(*) as total FROM appointments WHERE appt_date >= DATE_SUB(CURDATE(), INTERVAL 14 DAY) GROUP BY appt_date ORDER BY appt_date`. In PHP, encode the result as JSON using `json_encode()`. Pass it to an inline `<script>` block that initialises a Chart.js bar chart. Chart.js is already included in AdminLTE.

15. Grading Rubric

Component	Marks
Project structure, .htaccess routing, partials	5%
Database schema (5 tables, FKs, seed data)	5%
Core classes: Singleton DB, Auth, CSRF, Paginator	9%
Models: all 5 models with correct prepared statements	10%
Authentication: login wall, session management, security	9%
Admin panel: user CRUD, doctor CRUD, specializations	9%
Appointments: book (with conflict check), list, filter, status update	15%
Prescriptions: add with file upload, secure file serving	9%
Dashboard stats: all 3 role dashboards with live data	5%
Reports with CSV export	5%
Security: CSRF, XSS, no raw SQL, file validation, role enforcement	9%
Code quality: comments, naming conventions, no dead code	5%
GitHub: meaningful commits, README, SQL export file	5%
TOTAL	100%

16. AI Integration Guide

Using AI as a learning tool, not a shortcut

This project is large enough that AI tools will feel very tempting to use as a code generator. This section explains exactly how to use them responsibly — in a way that builds real skill instead of creating the illusion of it.

One practical rule: never paste more than 5 lines of AI output into your project without being able to explain every line immediately, out loud, without looking at the AI response. If you cannot — delete it and ask AI to explain the concept instead.

Step 1 — Architecture and class design

Before writing code, build a clear mental picture of how the pieces connect. Use AI to test your understanding.

Prompts:

Explain the Singleton pattern to me with a database connection as the example. Do not write PHP code — explain the concept in plain English and tell me what problem it solves.

I am building a PHP project with this folder structure: [paste]. My index.php will route requests using `$_GET["page"]` and `$_GET["action"]`. Explain this pattern to me — what are its advantages and what are its limitations?

What is the difference between a Model class and a Controller in PHP without a framework? Give me two concrete examples of logic that belongs in each.

Step 2 — Building the Database class and prepared statements

Write your Database class yourself. Use AI only to verify your understanding of the concepts involved.

Prompts:

How does `bind_param()` work in PHP mysqli? Explain the type string ("ssi" etc.) and what happens internally when I call it.

I wrote this Singleton class: [paste your attempt]. Is my `getInstance()` method correctly preventing multiple instances? What would happen if two models called `getInstance()` simultaneously?

My prepared statement returns false but no error is shown. I have this code: [paste]. What are the three most common reasons a PHP prepared statement fails silently?

What does `get_result()` return for a SELECT query vs an INSERT query? Why do I need to handle them differently?

Step 3 — Authentication flow and sessions

Security decisions in auth are easy to get subtly wrong. Use AI to understand WHY each step is required.

Prompts:

Why do I need to call `session_regenerate_id(true)` after a successful login? What specific attack does this prevent? Explain the attack scenario.

My login form submits but the session data disappears on the next page. I call `session_start()` at the top of every file. Here is my login handler: [paste]. What am I likely missing?

Explain why "Invalid credentials" is a better error message than showing "Email not found" or "Wrong password" separately. What attack does the generic message prevent?

What is the difference between `session_unset()` and `session_destroy()`? Which one do I call, in what order, during logout?

Step 4 — Dynamic WHERE clauses and complex queries

The appointment filter system requires building SQL dynamically. Understand the pattern before writing it.

Prompts:

I need to build a SQL WHERE clause in PHP where the user may provide 0 to 4 filter values (status, doctor_id, start_date, end_date). If no filters are given I want all records. Explain the approach of using a `$conditions` array without writing the code — I want to understand the pattern first.

How does `LIMIT ? OFFSET ?` work in SQL? If I am on page 3 and showing 10 items per page, what is the `OFFSET` value and how do I calculate it?

Explain what a SQL JOIN does between appointments and users tables. I need to show the patient name and doctor name on each appointment row. Describe in plain English what the query structure looks like.

I want to count appointments grouped by status in one query. Explain `GROUP BY` and `COUNT(*)` — what does the result set look like and how do I loop through it in PHP?

Step 5 — File upload and secure serving

File handling has two parts: validating the upload and controlling access after. Both are security-critical.

Prompts:

Why is checking `$_FILES["file"]["type"] === "application/pdf"` not a reliable security check? What can an attacker do to bypass it and what should I use instead?

My uploaded file is saving to disk but with 0 bytes. My form has `enctype="multipart/form-data"` on the `<form>` tag. Here is my upload code: [paste]. What else could cause this?

I want to serve a PDF file through PHP so that users cannot access it directly via URL. Explain what `readfile()` does and what HTTP headers I need to set before calling it.

Explain `getimagesize()` — how does it work, what does it return, and why is it more reliable than checking the file extension for image uploads?

Step 6 — Role-based access and ownership checks

Two different concepts: role checks (are you a doctor?) and ownership checks (is this YOUR appointment?). Both are required.

Prompts:

What is the difference between authentication and authorization? Give me a concrete example from a clinic management system.

I am checking roles by hiding menu items in the HTML for roles that should not see them. My instructor said this is not secure enough. Why? What does "enforce authorization server-side" mean in practice?

A patient is trying to access `?page=prescription&action=download&id=5`. The prescription belongs to a different patient. My code checks `Auth::requireRole("patient")` but not ownership. What query should I run to verify the requester owns this resource?

Step 7 — Code review and improvement

After each Part is working, request a review. Treat AI feedback as you would a senior developer's pull request comment.

Prompts:

Here is my `AppointmentModel::getByDoctor()` method: [paste]. Review it as a senior PHP developer. Do not rewrite it — identify: any SQL injection risk, any missing edge case, any OOP principle I am violating.

My `AppointmentController` is 200 lines long. Here it is: [paste]. A senior developer said I should "extract responsibilities into smaller methods." Which specific blocks in my code would they be referring to and why?

I want to add meaningful PHP docblock comments to my model methods. Show me one correct example of a docblock for a method with parameters and a return type, then I will write the rest myself.

The three habits: (1) Attempt before asking. (2) Explain back before moving on. (3) Never paste what you cannot defend. These three habits are the difference between a developer who grows and one who stagnates.

Good luck 🙌